nacap

# Implementación, Pruebas y Calidad de *Software*



# Tabla de CONTENIDOS


Introducción..... 03

**Introducción a SaaS, IaaS y *cloud computing***
¿Qué es *cloud computing*?..... 04
¿Qué es SaaS?..... 05
¿Qué es IaaS?..... 06
Relevancia en proyectos de *software*..... 06
Recursos complementarios..... 07
Cierre..... 07
Ideas fuerza..... 07

**Sistemas distribuidos y mensajería P2P**
¿Qué es un sistema distribuido?..... 08
Tipos de sistemas distribuidos..... 09
Recursos complementarios..... 10
Cierre..... 11
Ideas fuerza..... 11

**Normativa y ética en la confiabilidad de *software***
¿Qué es la ética en el uso del *software*?..... 12
Importancia de la confiabilidad para los negocios..... 13
¿Qué es el delito informático?..... 13
Marco Legal: Ley 21459 sobre delitos..... 13
informáticos en Chile..... 13
Recursos complementarios..... 13
Cierre..... 14
Ideas fuerza..... 14

**Desarrollo de prototipos según requisitos de proyecto**
¿Qué es un prototipo?..... 15
Proceso de desarrollo del prototipo según requisitos..... 15
Buenas prácticas OWASP en el desarrollo de prototipos..... 16
Normativa sobre delitos informáticos..... 16
Recursos complementarios..... 16
Cierre..... 17
Ideas fuerza..... 17

**Modelado de sistemas mediante diagramas UM**
Principales estándares de calidad en *software*..... 18
Pasos para crear diagramas UML en el prototipo..... 19
Recursos complementarios..... 19
Cierre..... 20
Ideas fuerza..... 20

**Pruebas de *testing* y estándares de calidad**
Pruebas de *testing*..... 21
Estándares de seguridad ISO 27000..... 21
Recursos complementarios..... 22

# Introducción

En este recurso, explorarás una variedad de temas esenciales para el desarrollo y la gestión de *software*. Comenzarás con los conceptos fundamentales del *cloud computing* y sus modelos de servicio, SaaS e IaaS, destacando sus beneficios y aplicaciones populares como Microsoft 365 y AWS.

Luego, te sumergirás en los sistemas distribuidos y la mensajería P2P, analizando sus características, ventajas y tendencias tecnológicas actuales, como los microservicios y la integración de IoT.

También abordarás la confiabilidad del *software* y su relación con la ética profesional y las normativas legales, como la Ley 21459 sobre delitos informáticos. Examinarás la responsabilidad ética de los desarrolladores y la importancia de cumplir con los códigos de ética del IEEE y la ACM.

El desarrollo de prototipos será otro tema clave, donde aprenderás sobre la definición de requerimientos, la construcción y validación de prototipos, y la incorporación de buenas prácticas de seguridad OWASP.

Además, te adentrarás en el modelado de sistemas con UML, explorando los diferentes tipos de diagramas y su aplicación práctica en proyectos de *software*.

Finalmente, revisarás las pruebas de *testing* y los estándares de calidad, incluyendo la norma ISO 27000, para asegurar la seguridad y calidad del *software*, y la importancia de documentar y auditar regularmente.

Esta unidad te proporcionará una visión integral y práctica de los aspectos clave en el desarrollo y la gestión de *software*, preparándote para enfrentar los desafíos y aprovechar las oportunidades en este campo dinámico.

# Introducción a SaaS, IaaS y *cloud computing*



En la actualidad, el panorama tecnológico está definido por una evolución constante hacia la digitalización y la adopción de soluciones basadas en la nube. Dentro de este contexto, el *cloud computing*, junto con sus principales modelos de servicio: SaaS *(Software as a Service)* e IaaS *(Infrastructure as a Service)* se han convertido en piezas clave para el desarrollo y la implementación de proyectos de *software.*

A continuación, exploraremos los conceptos fundamentales de estas tecnologías y su aplicación en proyectos de software modernos.

## ¿Qué es *cloud computing*?

*Cloud computing*, o computación en la nube, se refiere a **la entrega de recursos informáticos**, como almacenamiento, potencia de cálculo y aplicaciones, a través de Internet. Este paradigma **permite a las organizaciones acceder a servicios y recursos bajo demanda, sin necesidad de realizar grandes inversiones en infraestructura física**. Los beneficios clave incluyen escalabilidad, flexibilidad, reducción de costos operativos y la capacidad de enfocarse en el desarrollo de su negocio principal en lugar de en la gestión de infraestructura.

**Beneficios del *cloud computing***

- **Escalabilidad:** ajuste rápido de recursos según demanda.
- **Flexibilidad:** acceso a recursos desde cualquier lugar.
- **Reducción de costos:** menores inversiones en infraestructura física.
- **Enfoque en negocio:** permite a las organizaciones centrarse en su actividad principal.

## ¿Qué es SaaS?

Es un modelo de servicio en el que las aplicaciones de *software* se ofrecen a los usuarios a través de Internet, eliminando la necesidad de instalar y mantener *software* localmente. Los proveedores de SaaS alojan y gestionan las aplicaciones en sus servidores, lo que permite a los usuarios acceder a ellas desde cualquier lugar con una conexión a Internet.

### ¿De dónde viene?

También conocido como *software* alojado o *software* basado en la web surge a finales de los años noventa con la promesa de **beneficiar a los negocios con costos más bajos, implementación rápida y escalabilidad.**

### Principales características de SaaS

- **Acceso bajo demanda:** los usuarios pueden acceder a las aplicaciones desde navegadores web o dispositivos móviles.
- **Actualizaciones automáticas:** los proveedores se encargan de mantener el *software* actualizado.
- **Modelos de suscripción:** generalmente, los servicios SaaS se ofrecen bajo un esquema de pago por uso o suscripción mensual/anual.

### Ejemplos de SaaS

- Herramientas de colaboración como Microsoft 365 o *Google Workspace.*
- Plataformas de gestión de proyectos como Trello o Asana.
- Soluciones de gestión de relaciones con clientes (CRM) como *salesforce.*




**Figura 1.** Ventajas del SaaS.

**Fuente.** CONTPAQI (S.f).

## ¿Qué es IaaS?

Es una forma de computación en la nube que ofrece recursos fundamentales de infraestructura de TI, como informática, servidores, máquinas virtuales, redes y almacenamiento a los consumidores, a través de Internet y de pago por uso. Este modelo **permite a las organizaciones alquilar recursos informáticos según sus necesidades, en lugar de comprarlos y gestionarlos físicamente**.

**Principales características de IaaS**

- **Escalabilidad:** los recursos pueden ajustarse rápidamente en función de las demandas del proyecto.
- **Pago por uso:** los usuarios solo pagan por los recursos que utilizan.
- **Control total:** permite a los desarrolladores personalizar y configurar entornos según sus requisitos específicos.

## Relevancia en proyectos de *software*

El uso de SaaS e IaaS dentro del marco de *cloud computing* ha revolucionado la forma en que las organizaciones abordan el desarrollo y la gestión de proyectos de *software*. Estos modelos permiten:

- **Agilidad:** implementar soluciones rápidamente sin necesidad de preocuparse por la infraestructura subyacente.
- **Colaboración:** facilitar la colaboración entre equipos distribuidos geográficamente.
- **Reducción de costos:** minimizar gastos en hardware y mantenimiento.
- **Innovación:** enfocarse en crear valor a través de nuevas funcionalidades y servicios.

**Tabla 1.** Comparación de modelos de servicio en la nube.

| Característica | SaaS | IaaS |
|---|---|---|
| Acceso | Navegadores *web*, dispositivos móviles | Virtualización de recursos |
| Gestión | Proveedor | Usuario |
| Pago | Suscripción | Pago por uso |
| Ejemplos | Microsoft 365, *Google Workspace* | AWS, Microsoft Azure |

**Fuente.** Elaboración propia.

## Recursos complementarios

*Blog*

**Nanobytes**

IaaS, PaaS y SaaS

Leer artículo: https://nanobytes.es/blog/blog-nanobytes-1/cloud-computing-diferencia-entre-iaas-paas-y-saas-7

*YouTube*

**EDteam**

¿Qué es la nube (cloud computing)? - La mejor explicación en español

Ver video: https://www.youtube.com/watch?v=h4Af5bbFAq0

*YouTube*

**KeepCoding - Tech School**

¿Qué es Iaas?

Ver video: https://www.youtube.com/watch?v=eXG-gRraXfY

## Cierre

La adopción de *cloud computing*, SaaS e IaaS está transformando el panorama tecnológico y empresarial.

Estos modelos de servicio permiten a las organizaciones acceder a recursos informáticos de manera flexible y escalable, reduciendo costos y mejorando la eficiencia operativa. La comprensión de estos conceptos es esencial para cualquier profesional del área de *software* que desee mantenerse competitivo en un entorno digital en constante evolución.

## Ideas fuerza

La implementación de SaaS e IaaS dentro del marco de *cloud computing* ofrece múltiples beneficios que están revolucionando la gestión y desarrollo de proyectos de *software*. Estos modelos no solo facilitan la agilidad y colaboración, sino que también permiten una reducción significativa de costos y un enfoque en la innovación.

**1. Escalabilidad y flexibilidad**

- Ajuste rápido de recursos según demanda.
- Acceso a recursos desde cualquier lugar con conexión a Internet.
- Adaptación a las necesidades cambiantes del proyecto.

**2. Reducción de costos**

- Menores inversiones en infraestructura física.
- Pago por uso, optimizando el gasto en recursos.
- Eliminación de costos de mantenimiento y actualización.

**3. Agilidad en la implementación**

- Implementación rápida de soluciones sin preocuparse por la infraestructura subyacente.
- Facilita la adopción de nuevas tecnologías y funcionalidades.
- Permite a las organizaciones centrarse en su actividad principal.

**4. Colaboración mejorada**

- Facilita la colaboración entre equipos distribuidos geográficamente.
- Herramientas de colaboración en tiempo real.
- Mejora la comunicación y coordinación entre los miembros del equipo.

**5. Innovación continua**

- Enfoque en crear valor a través de nuevas funcionalidades y servicios.
- Acceso a las últimas tecnologías y actualizaciones.
- Fomenta un entorno de desarrollo ágil y dinámico.

# Sistemas distribuidos y mensajería P2P




## ¿Qué es un sistema distribuido?

Un sistema distribuido es un conjunto de programas informáticos que utilizan recursos computacionales en varios nodos de cálculo distintos para lograr un objetivo compartido común. Este tipo de sistemas, también denominados **computación distribuida o bases de datos distribuidas**, usan nodos distintos para comunicarse y sincronizarse a través de una red común.

Estos nodos suelen representar dispositivos de *hardware* físicos diferentes, pero también pueden representar procesos de *software* diferentes u otros sistemas encapsulados recursivos.

Los sistemas de computación distribuida se caracterizan por:

- **Recursos compartidos:** *hardware, software* o datos.
- **Procesamiento simultáneo:** varias máquinas procesan la misma función a la vez.
- **Escalabilidad:** capacidad de aumentar recursos al añadir más máquinas.
- **Detección de errores:** errores detectados más fácilmente.
- **Transparencia:** nodos pueden acceder y comunicarse entre sí.

## Tipos de sistemas distribuidos

**1. *Cloud computing* (Computación en la nube)**

Se refiere a la entrega de servicios de computación a través de Internet. Incluye almacenamiento, servidores, bases de datos, redes y *software*, proporcionando acceso a través de Internet en lugar de tener recursos locales.

**2. *Grid computing***

El *grid computing* o computación en malla apareció como una respuesta a la necesidad de contar con ordenadores más potentes que pudieran realizar tareas más complejas, como puede ser el ejecutar cálculos muy amplios.

Se trata de unir a varios procesadores en una única tarea conectados a una misma red de comunicación o a Internet y, generalmente, se conoce como procesamiento en paralelo.

En la computación en malla o de rejilla puede ocurrir que todos los recursos de esa red se utilicen por todos los usuarios, o que algunos tengan restricciones de algún tipo.

**3. *Cluster computing***

Consiste en **un grupo de computadoras conectadas que trabajan juntas como un único sistema**. Se enfoca en proporcionar alta disponibilidad y rendimiento mejorado, ideal para aplicaciones que requieren procesamiento intensivo, como la minería de datos.

**4. Mensajería P2P *(Peer-to-Peer)***

Un modelo de comunicación en el que cada participante (o peer) actúa tanto como cliente como servidor. Esto permite **una mayor escalabilidad y redundancia**, ya que no hay un punto único de fallo.

**En resumen**

Los sistemas distribuidos están compuestos por múltiples nodos interconectados que trabajan de manera coordinada para proporcionar un servicio.

**Tabla 2.** Modelos de sistemas distribuidos.

| Modelo | Descripción | Ejemplo | Ventajas | Desventajas |
|---|---|---|---|---|
| ***Cloud computing*** | Uso de recursos distribuidos en la nube con alta escalabilidad y elasticidad. | Dropbox permite a los usuarios almacenar y sincronizar archivos en la nube, accesibles desde cualquier dispositivo. | Alta escalabilidad, acceso global, reducción de costos en *hardware*. | Dependencia de proveedores, posibles problemas de seguridad y privacidad. |
| ***Grid computing*** | Computación en malla, donde múltiples computadoras colaboran para resolver problemas de alto procesamiento. | La red de investigación SETI@home usa computadoras de voluntarios para analizar señales del espacio en busca de inteligencia extraterrestre. | Alto poder de procesamiento, aprovechamiento de recursos distribuidos. | Complejidad en la administración, latencia en la comunicación entre nodos. |
| ***Cluster computing*** | Conjunto de servidores agrupados que trabajan juntos para mejorar el rendimiento y la disponibilidad. | Un banco usa un clúster de servidores para procesar transacciones de clientes en tiempo real. | Alta disponibilidad, redundancia y balanceo de carga. | Costo elevado en *hardware* y mantenimiento. |
| **Mensajería P2P** | Comunicación directa entre dispositivos sin intermediarios, utilizada en aplicaciones como torrents y VoIP. | Skype permite llamadas de voz y video sin pasar por servidores centrales, gracias a su tecnología P2P. | Distribución descentralizada, resistencia a fallos. | Difícil control y monitoreo, vulnerabilidades de seguridad. |

**Fuente.** Elaboración propia.

## Recursos complementarios

*YouTube*

**Mick Hacking**

Redes Peer to Peer ( P2P ) | La mejor explicación

Ver video: https://www.youtube.com/watch?v=iP28dHZIgBc

*Blog*

**IONOS Digital guide**

¿Qué es P2P (peer to peer)?

Leer artículo: https://www.ionos.com/es-us/digitalguide/servidores/know-how/que-es-p2p-peer-to-peer/?srsltid=AfmBOorsg6uXr8_-g2bPgPDmPEGFGT5OJ40QMTEOD4A6Hm0fqip0ZR2V

*Blog*

**IONOS Digital guide**

¿Qué es el Grid Computing?

Leer artículo: https://www.ionos.com/es-us/digitalguide/servidores/know-how/grid-computing/?srsltid=AfmBOoqUo8KhisY-QXKuknsgvjqMD8UIIISy-VP21RmW_4VwJogUc7MMm

## Cierre

Los sistemas distribuidos y la mensajería P2P son fundamentales para las innovaciones tecnológicas actuales.

Has explorado los conceptos y aplicaciones de los sistemas distribuidos y la mensajería P2P. Los sistemas distribuidos permiten compartir recursos y procesar tareas simultáneamente, mejorando la escalabilidad y la detección de errores. La mensajería P2P, por su parte, facilita la comunicación directa entre dispositivos, aumentando la resiliencia y reduciendo costes.

Además, has visto cómo la computación en la nube, la computación en malla y la computación en clúster son ejemplos de sistemas distribuidos que ofrecen diversas ventajas para las empresas y los usuarios. También has discutido las tendencias tecnológicas actuales, como el aumento en el uso de SaaS, microservicios y contenedores, y la integración de IoT.

## Ideas fuerza

En resumen, los sistemas distribuidos y la mensajería P2P son pilares esenciales en el desarrollo de tecnologías modernas. Su capacidad para mejorar la escalabilidad, eficiencia y resiliencia los convierte en herramientas indispensables para empresas y desarrolladores.

Mantenerse actualizado con las tendencias tecnológicas y comprender estos conceptos es crucial para aprovechar al máximo sus beneficios en un entorno en constante evolución.

**1. Escalabilidad de los sistemas distribuidos**

- Permiten añadir más nodos para aumentar la capacidad de procesamiento.
- Facilitan el manejo de grandes volúmenes de datos y usuarios.
- Mejoran la eficiencia operativa en entornos empresariales.

**2. Ventajas de la mensajería P2P**

- Reducción de costes al eliminar la necesidad de servidores centrales.
- Mayor eficiencia y velocidad en la transferencia de datos.
- Alta escalabilidad y resiliencia ante fallos.

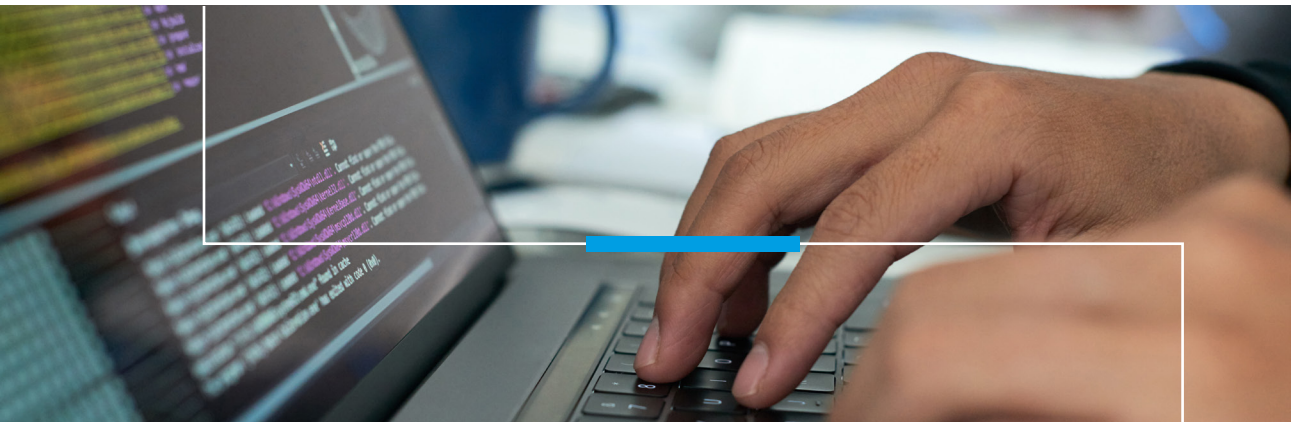
**3. Computación en la nube**

- Proporciona acceso a recursos de infraestructura bajo demanda.
- Modelos de servicio como SaaS, IaaS y PaaS ofrecen flexibilidad y ahorro de costes.
- Facilita el desarrollo y despliegue de aplicaciones sin preocuparse por la infraestructura subyacente.

**4. Tendencias tecnológicas**

- Aumento en el uso de SaaS para reducir costos operativos.
- Popularidad de microservicios y contenedores para un desarrollo ágil.
- Integración de IoT en sistemas distribuidos para manejar la comunicación entre dispositivos.

# Normativa y ética en la confiabilidad de *software*



## ¿Qué es la ética en el uso del *software*?

Implica un proceso meticuloso de planificación, diseño, implementación, prueba y mantenimiento para crear sistemas confiables, eficientes y seguros. Sin embargo, además de buscar funcionalidad y rendimiento, los desarrolladores también deben considerar cuidadosamente los aspectos éticos del *software* que están creando.

En el ámbito de la programación, constantemente se promueve la innovación y desarrollando soluciones que cambian la interacción con el mundo digital.

## Importancia de la confiabilidad para los negocios

La confiabilidad en el *software* tiene implicaciones directas en la eficiencia operativa, la satisfacción del cliente y la reputación de las empresas. Para los negocios, un *software* confiable significa:

**1. Reducción de riesgos:** sistemas poco confiables pueden provocar pérdida de datos, interrupciones en las operaciones y riesgos financieros significativos.

**2. Mejora de la competitividad:** empresas que entregan soluciones tecnológicas confiables ganan la preferencia de los clientes y se posicionan mejor en el mercado.

**3. Cumplimiento normativo:** al operar con software confiable, las empresas disminuyen la probabilidad de infringir leyes o regulaciones relacionadas con la privacidad y seguridad de los datos.

## ¿Qué es el delito informático?

El término 'delito informático' abarca una amplia variedad de actividades delictivas que se realizan utilizando computadoras, redes u otros dispositivos digitales. Este término incluye diversas acciones ilegales llevadas a cabo por delincuentes informáticos, tales como la piratería informática, el *phishing*, el robo de identidad, el r*ansomware* y los ataques de *malware*, entre otros.

## Marco Legal: Ley 21459 sobre delitos informáticos en Chile

La Ley 21459 en Chile establece un marco integral para prevenir y sancionar los delitos informáticos, alineándose con el Convenio de Budapest. Penaliza accesos no autorizados a información sensible y actos que comprometan la funcionalidad de sistemas informáticos, como la introducción de malware. Las empresas deben implementar medidas de seguridad y cooperar con las autoridades en investigaciones penales. El cumplimiento de esta ley es esencial para mantener la confianza en las plataformas digitales y proteger contra amenazas cibernéticas.

## Recursos complementarios

*YouTube*

**Facultad de Derecho UChile**

Ley N° 21.459 sobre Delitos Informáticos

Ver video: https://www.youtube.com/watch?v=kXprEgw54Uc

*Página web*

**Association for Computing Machinery**

Código de Ética y Conducta Profesional de ACM

Leer artículo: https://www.acm.org/about-acm/code-of-ethics-in-spanish

## Cierre

La confiabilidad del software y la ética profesional son esenciales para el éxito empresarial y la seguridad informática.

En este documento, has explorado la importancia de la confiabilidad del software en los negocios y su relación con la ética profesional y las normativas legales.

La confiabilidad del software no solo reduce riesgos y mejora la competitividad, sino que también asegura el cumplimiento normativo, protegiendo tanto a las empresas como a los usuarios. Además, la ética en el desarrollo de software es fundamental para garantizar la calidad, seguridad y transparencia en el uso de sistemas tecnológicos.

La Ley 21459 sobre delitos informáticos en Chile establece un marco legal integral para prevenir y sancionar los delitos informáticos, promoviendo la seguridad y confiabilidad en el uso de sistemas tecnológicos. Las empresas tienen la responsabilidad de implementar medidas de seguridad adecuadas y cooperar con las autoridades para proteger la información sensible y evitar sanciones por negligencia.

## Ideas fuerza

En resumen, la confiabilidad del *software* y la ética profesional son pilares fundamentales para el desarrollo de sistemas tecnológicos seguros y eficientes. Las empresas deben adoptar prácticas éticas y cumplir con las normativas legales para proteger a los usuarios y mantener la confianza en sus plataformas digitales.

**1. Confiabilidad del *software***

- Reduce riesgos operativos y financieros.
- Mejora la competitividad empresarial.
- Asegura el cumplimiento normativo.

**2. Ética profesional en el desarrollo de *software***

- Garantiza la calidad y seguridad del *software*.
- Protege la privacidad y seguridad de los usuarios.
- Promueve la transparencia y la integridad.

**3. Ley 21459 sobre delitos informáticos**

- Penaliza accesos no autorizados y sabotajes digitales.
- Establece responsabilidades corporativas en ciberseguridad.
- Fomenta la cooperación entre empresas y autoridades.

# Desarrollo de prototipos según requisitos de proyecto



## ¿Qué es un prototipo?

Es una versión inicial de un sistema de software que **permite visualizar el producto final**, identificar problemas tempranos y validar y refinar los requerimientos con los interesados.

**Tipos de prototipos**

**1. Prototipo de baja fidelidad**

- **Características:** bocetos y *wireframes* que muestran la estructura y flujo general del sistema.

**2. Prototipo funcional o de alta fidelidad**

- **Características:** Interactividad, detalles visuales refinados y funcionalidades básicas.

## Proceso de desarrollo del prototipo según requisitos

El desarrollo de un prototipo efectivo sigue un enfoque estructurado dividido en etapas clave:

**1. Definición de requerimientos:**

- **Funcionales:** qué debe hacer el sistema (ej., registro de usuarios, búsqueda en tiempo real).
- **No funcionales:** rendimiento, usabilidad, seguridad (ej., respuesta rápida, estándares de accesibilidad).

**2. Consulta con los interesados:**

- Entrevistas y encuestas para entender necesidades.

**3. Documentación de requerimientos:**

- Uso de documentos como SRS o historias de usuario.
- Clasificación de requisitos por prioridad: esenciales, deseables y opcionales.

## Buenas prácticas OWASP en el desarrollo de prototipos

La *Open Web Application Security Project* (OWASP) recomienda buenas prácticas para la seguridad en el desarrollo de software, como:

- **Validación de entrada:** prevenir inyecciones de código.
- **Gestión de autenticación y sesiones:** usar contraseñas fuertes, *hashing* seguro y MFA.
- **Protección de datos sensibles:** usar cifrado TLS y evitar almacenar datos innecesarios.
- **Manejo de errores:** no revelar detalles del sistema y registrar errores de manera segura.
- **Actualización y mantenimiento:** usar bibliotecas actualizadas y monitorear vulnerabilidades.

## Normativa sobre delitos informáticos

Cumplir con las normativas legales y éticas es esencial para evitar riesgos legales en el desarrollo de software. Entre las normativas relevantes están:

- **Ley de delitos informáticos** (dependiendo de tu país, puedes investigar la legislación específica).
- **Reglamento General de Protección de Datos** (GDPR) para el manejo de datos personales si el *software* opera en la UE.
- **Ley de derechos de autor** para respetar licencias de *software* y contenido.

## Recursos complementarios

*Página web*

**Biblioteca del Congreso Nacional de Chile**

Delitos informáticos

Leer artículo: https://www.bcn.cl/portal/leyfacil/recurso/delitos-informaticos

*Página web*

**Hostingplus**

Modelo de prototipos: ¿qué es y cuáles son sus etapas?

Leer artículo: https://www.hostingplus.pe/blog/modelo-de-prototipos-que-es-y-cuales-son-sus-etapas/

## Cierre

El desarrollo de prototipos es esencial para validar ideas y garantizar la calidad y seguridad del *software*.

El desarrollo de prototipos es una fase estratégica en la creación de *software*, permitiendo validar ideas, entender los requerimientos y ajustar funcionalidades antes de implementar un sistema completo. Este proceso, cuando se realiza de manera estructurada y alineada con las buenas prácticas de desarrollo OWASP, asegura que el producto final cumpla con los estándares de calidad y seguridad necesarios.

Incorporar buenas prácticas de seguridad desde el inicio y cumplir con las normativas legales y éticas no solo protege a los usuarios y al negocio de posibles riesgos, sino que también refleja un enfoque profesional y ético en el desarrollo de *software*. La validación constante y la retroalimentación de los usuarios son cruciales para asegurar que el prototipo cumpla con los objetivos del proyecto.

## Ideas fuerzas

El desarrollo de prototipos no solo facilita la validación de ideas y la identificación temprana de problemas, sino que también asegura que el producto final cumpla con los estándares de calidad y seguridad necesarios. Al seguir un proceso estructurado y alineado con las buenas prácticas de desarrollo OWASP, y al cumplir con las normativas legales, se protege tanto a los usuarios como al negocio, reflejando un enfoque profesional y ético en el desarrollo de *software*.

**1. Validación de ideas y requerimientos**

- Permite visualizar cómo funcionará el producto final.
- Identifica posibles problemas en etapas tempranas.
- Valida y refina los requerimientos junto con los interesados.

**2. Proceso estructurado**

- Definición clara de requerimientos funcionales y no funcionales.
- Análisis y priorización de sistemas críticos.
- Diseño y construcción iterativa del prototipo.

**3. Buenas prácticas de seguridad OWASP**

- Validación de todas las entradas del usuario.
- Gestión segura de autenticación y sesiones.
- Protección de datos sensibles mediante cifrado.

**4. Cumplimiento de normativas legales**

- Adherencia a la Ley de Delitos Informáticos y GDPR.
- Respeto por las licencias de software y contenido.

**5. Retroalimentación y mejora continua**

- Pruebas de usuario y funcionalidad para asegurar usabilidad.
- Integración de cambios sugeridos por los usuarios y partes interesadas.

# Modelado de sistemas mediante diagramas UM



## Principales estándares de calidad en *software*

El modelado de sistemas mediante diagramas UML *(Unified Modeling Language)* es una práctica esencial en **el diseño y desarrollo de sistemas de *software***. Este enfoque permite representar gráficamente la estructura, el comportamiento y las interacciones del sistema, facilitando la comprensión, comunicación y colaboración entre los miembros de un equipo.

**Tipos de diagramas UML utilizados en un prototipo**

**1. Diagramas de estructura**

- **Diagrama de clases:** representa clases, atributos, métodos y relaciones.
- **Diagrama de componentes:** muestra la interacción entre componentes.
- **Diagrama de despliegue:** representa la arquitectura física del sistema.

**2. Diagramas de comportamiento:**

- **Diagrama de casos de uso:** identifica funcionalidades desde la perspectiva del usuario.
- **Diagrama de actividades:** representa el flujo de trabajo o procesos.

**3. Diagramas de interacción:**

- **Diagrama de secuencia:** representa el flujo de mensajes en orden cronológico.
- **Diagrama de colaboración:** enfocado en interacciones y relaciones entre objetos.

## Pasos para crear diagramas UML en el prototipo

La creación de diagramas UML para un prototipo **requiere un enfoque estructurado** que combine la comprensión del sistema a modelar con el uso de herramientas y metodologías apropiadas.

A continuación, se detalla cada paso para garantizar que los diagramas sean efectivos y relevantes:

**1. Identificar los requisitos del sistema**

Antes de comenzar con cualquier diagrama, es crucial comprender los objetivos y las necesidades del sistema.

**2. Seleccionar los tipos de diagramas apropiados**

Dependiendo de las características del sistema y el objetivo del prototipo, se deben elegir los diagramas más adecuados.

**3. Asignar roles y funciones en el equipo**

El éxito en la creación de diagramas depende de un trabajo en equipo eficiente.

**4. Construir los diagramas**

Esta etapa implica la creación propiamente dicha de los diagramas, siguiendo las normas de UML.

**5. Validar los diagramas**

La validación asegura que los diagramas representen correctamente los requisitos y funcionen como una guía para el desarrollo.

**6. Documentar y entregar los diagramas**

Finalmente, los diagramas deben ser organizados y presentados como parte de la documentación del prototipo.

## Recursos complementarios

*YouTube*

**Universitat Politècnica de València - UPV**

Diagramas UML Estructurales para la Ingeniería del Software. | 1/38 | UPV

Ver video: https://www.youtube.com/watch?v=L8PAgQgDeMY

*Documento*

**Popkin Software and Systems**

Modelado de Sistemas com UML

Leer documento: http://es.tldp.org/Tutoriales/doc-modelado-sistemas-UML/doc-modelado-sistemas-uml.pdf

## Cierre

El uso de diagramas UML en el modelado de sistemas permite construir prototipos sólidos y bien estructurados.

Al aplicar roles claros, metodologías efectivas y un enfoque basado en los desafíos del equipo, se pueden lograr representaciones fieles y útiles del sistema.

El modelado de sistemas mediante diagramas UML es una práctica esencial en el diseño y desarrollo de *software*. Este enfoque facilita la comprensión, comunicación y colaboración entre los miembros del equipo, permitiendo representar gráficamente la estructura, el comportamiento y las interacciones del sistema.

Para crear diagramas UML efectivos, es crucial seguir un enfoque estructurado que incluya la identificación de requisitos, la selección de diagramas apropiados, la asignación de roles en el equipo, la construcción y validación de los diagramas, y finalmente, la documentación y entrega de estos. Este proceso asegura que los diagramas sean relevantes y útiles para el desarrollo del sistema.

## Ideas fuerza

En resumen, el uso de diagramas UML en el modelado de sistemas es una herramienta poderosa para construir prototipos bien estructurados y efectivos. Al seguir un enfoque estructurado y colaborativo, se pueden lograr representaciones fieles del sistema que faciliten su desarrollo y optimización.

**1. Claridad en la representación del sistema**

- Proporciona una visión clara de los componentes del sistema y sus relaciones.
- Facilita la comprensión del sistema por parte de todos los miembros del equipo.
- Ayuda a identificar posibles mejoras y optimizaciones.

**2. Mejora de la colaboración**

- Mejora la comunicación entre desarrolladores, diseñadores y clientes.
- Permite una mejor coordinación y distribución de tareas dentro del equipo.
- Facilita la integración de diferentes perspectivas y conocimientos.

**3. Adaptabilidad a los desafíos del proyecto**

- Permite ajustar roles, funciones y metodologías según las necesidades del proyecto.
- Facilita la incorporación de cambios y nuevas funcionalidades.
- Ayuda a gestionar las limitaciones del sistema, como restricciones de tiempo y presupuesto.

**4. Eficiencia en el desarrollo del sistema**

- Asegura que los diagramas UML sean efectivos y relevantes.
- Reduce el riesgo de errores y retrabajos durante el desarrollo.
- Proporciona una guía clara para la implementación del sistema.

# Pruebas de *testing* y estándares de calidad



El proceso de *testing* y la aplicación de estándares de calidad son fundamentales para garantizar el éxito de un prototipo de desarrollo.

## Pruebas de *testing*

**¿Qué son?**

Son un conjunto de actividades diseñadas para **garantizar la calidad, funcionalidad y seguridad de un sistema**.

## Estándares de seguridad ISO 27000

Los estándares de calidad y seguridad de *software* establecen directrices para asegurar la correcta gestión, desarrollo y mantenimiento de *software* con criterios de calidad y seguridad adecuados.

**Tabla 3.** Estándares de calidad y seguridad de *software*.

| Estándar | Descripción | Ejemplo de uso |
|---|---|---|
| **ISO 12207** | Define procesos para el ciclo de vida del *software*, desde la planificación hasta el mantenimiento. | Empresas de desarrollo estructuran sus procesos de planificación, diseño, desarrollo y pruebas para asegurar la calidad y sostenibilidad del *software*. |
| **ISO 33000 (ex 15504)** | Evalúa la madurez de los procesos de desarrollo de *software.* | Una empresa que busca certificación en calidad de software utiliza esta norma para medir y mejorar sus procesos de desarrollo. |
| **ISO 9001** | Marco para la gestión de calidad en cualquier industria, incluyendo el desarrollo de *software.* | Una empresa de software implementa ISO 9001 para garantizar que su software cumple con requisitos de clientes y mejora continuamente sus procesos. |
| **ISO 25000** | Establece características para evaluar la calidad del *software*, como usabilidad, eficiencia y mantenibilidad. | Un equipo de desarrollo utiliza ISO 25000 para garantizar que una aplicación móvil tenga alta usabilidad y rendimiento. |
| **ISO 27000** | Define estándares para la gestión de la seguridad de la información en sistemas informáticos. | Empresas que manejan datos sensibles, como bancos o aseguradoras, implementan ISO 27000 para proteger datos de clientes contra amenazas cibernéticas. |

**Fuente.** Elaboración propia.

## Recursos complementarios

*Página web*

**Global suite**

ISO 27000 y el conjunto de estándares de Seguridad de la Información

Leer blog: https://www.globalsuitesolutions.com/es/la-familia-de-normas-iso-27000/

*Página web*

**Pym**

Los diferentes tipos de testing en el desarrollo de *software*

Leer artículo: https://programacionymas.com/blog/tipos-de-testing-en-desarrollo-de-software

# Bibliografía


Abadi, D. J. (2020). *Database systems: The complete book*. Prentice Hall.

Apache Software Foundation. (2023). *Apache JMeter Documentation*. Recuperado de https://jmeter.apache.org

Association for Computing Machinery. (n.d.). *Código de ética y conducta profesional de ACM.* Recuperado de https://www.acm.org/about-acm/code-of-ethics-in-spanish

Buyya, R., & Mehta, A. (2022). *Cloud computing: Principles and paradigms* (2nd ed.). Wiley.

CONTPAQI. (s.f.). *Software as a service: ¿Qué es?* Recuperado de https://www.contpaqi.com/publicaciones/gestion-empresarial/software-as-a-service-que-es

Google Cloud. (n.d.). *PaaS vs IaaS vs SaaS: What's the difference?* Recuperado de https://cloud.google.com

Hernández, K. (2022, septiembre 1). *Grid computing: ¿Qué es y cuáles son sus características?* Recuperado de https://www.servnet.mx/blog/grid-computing-que-es-y-cuales-son-sus-caracteristicas

IEEE. (n.d.). *Código de ética del IEEE*. Recuperado de https://www.ieee.org/about/corporate/governance/p7-8.html

IEEE Computer Society. (2018). *Standard for a pervasive computing reference model (IEEE Std 2700-2018)*. https://doi.org/10.1109/IEEESTD.2018.8589919

International Organization for Standardization. (2022). ISO/IEC 27001:2022 - *Information security, cybersecurity and privacy protection - Information security management systems - Requirements.* ISO.

Jamsa, K. (2013). *Cloud computing: SaaS, PaaS, IaaS, virtualization, business models, mobile, security and more*. Jones & Bartlett Publishers.

Kothapalli, M. (2018). *Cloud computing architectures: Comparing service models (IaaS, PaaS, SaaS) and deployment models (public, private, hybrid, community)—Uses and trade-offs*. Journal of Scientific and Engineering Research, 5(10), 334-341.

Kumar, S., & Kumar, A. (2021). *Cloud computing and its applications: An overview.* International Journal of Information Management, 57, 102-114. https://doi.org/10.1016/j.ijinfomgt.2021.102114

Naciones Unidas. (2000). *Convención sobre el Ciberdelito (Convención de Budapest).* Recuperado de https://www.coe.int/en/web/cybercrime

National Institute of Standards and Technology (NIST). (2018). *Framework for Improving Critical Infrastructure Cybersecurity (Version 1.1)*. Recuperado de https://www.nist.gov

OWASP Foundation. (2021). *OWASP Top Ten: The ten most critical web application security risks.* Recuperado de https://owasp.org

OWASP Foundation. (2023). *OWASP Testing Guide v4*. Recuperado de https://owasp.org

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach (9.ª ed.)*. McGraw-Hill Education.

República de Chile. (2022). *Ley 21459 sobre delitos informáticos*. Diario Oficial de la República de Chile. Recuperado de https://www.bcn.cl/leychile/navegar?idNorma=1177492

Sommerville, I. (2019). *Software Engineering (10.ª ed.)*. Pearson.

Sommerville, I. (2020). *Engineering software products: An introduction to modern software engineering (2.ª ed.)*. Pearson.

Susnjara, S., & Smalley, I. (2024, enero 22). *¿Qué es la infraestructura como servicio (IaaS)?* Recuperado de https://www.ibm.com/mx-es/topics/iaas

Tanenbaum, A. S. (2007). *Distributed systems: Principles and paradigms*. Prentice Hall.

Unión Europea. (2016). *Reglamento General de Protección de Datos (GDPR). Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo*. Recuperado de https://gdpr-info.eu

Usability.gov. (2021). *Prototyping and usability testing*. Recuperado de https://www.usability.gov

Vigna, G., & Monga, A. (2018). *Peer-to-peer systems: Principles and applications.* Springer.

World Wide Web Consortium (W3C). (2018). *Web content accessibility guidelines (WCAG) 2.1*. Recuperado de https://www.w3.org/TR/WCAG21

Zendesk. (2021, agosto 13). *¿Qué es un cluster y para qué sirve?* Recuperado de https://www.zendesk.com.mx/blog/cluster-que-es/